Arrays & Hashing: 10 NeetCode Problems

Oct 28, 2026 · @Meera Ramesh

Pattern map

All 10 problems come down to five patterns. Before coding a new problem, find its row here by asking the question in the last column.

#	Problem	Pattern	Time	Space	Recognize it when you ask…
1	Contains Duplicate	Seen before? (Set)	O(n)	O(n)	"Have I met this exact value already?"
2	Valid Anagram	Count and compare (Map)	O(n)	O(1)*	"Do both sides have the same amount of each thing?"
3	Two Sum	Seen before? (complement Map)	O(n)	O(n)	"Has the partner I need already shown up?"
4	Group Anagrams	Group by a key (Map of lists)	O(n·k log k)	O(n·k)	"What key makes all matching items land in one bucket?"
5	Top K Frequent Elements	Count, then rank	O(n log n)	O(n)	"Which items have the biggest counts?"
6	Encode and Decode Strings	Length-prefix encoding	O(m)	O(m)	"How does the reader know where each piece ends?"
7	Valid Palindrome	Two pointers	O(n)	O(1)	"Can I compare from both ends toward the middle?"
8	Product of Array Except Self	Prefix / suffix	O(n)	O(n)	"Is my answer everything to my left combined with everything to my right?"
9	Valid Sudoku	Seen before? across several groups	O(1)**	O(1)**	"Does each item belong to several groups that each need unique members?"
10	Longest Consecutive Sequence	Streak start (Set)	O(n)	O(n)	"Can I count each run only from its first item?"

* 26 lowercase letters at most. ** The board is always 9×9. n = number of items, k = longest word, m = total characters.

1. Contains Duplicate

Pattern: Seen before? (Set)

Like you're 10: You're a bouncer with a guest list. Each person who walks up, you check the list. If their name is already on it, you've caught a duplicate. If not, you write their name down and let them in.

Steps:

1. Make an empty Set.
2. For each number, check whether the Set has it. If yes, return `true`.
3. Otherwise add it to the Set.
4. If the loop finishes, return `false`.

```
var containsDuplicate = function(nums) {
    const seen = new Set();
    for (const num of nums) {
        if (seen.has(num)) return true;
        seen.add(num);
    }
    return false;
};
```

Complexity: O(n) time, O(n) space.

Watch out: check the value (`nums[i]`), not the index (`i`). Use a Set, not a Map, because you never need to remember anything extra about each number. The one-liner `new Set(nums).size !== nums.length` also works, but it can't stop early.

2. Valid Anagram

Pattern: Count and compare (Map)

Like you're 10: `s` fills a bag with letter tiles. `t` takes tiles out, one per letter. If `t` reaches for a tile that isn't in the bag, it's not an anagram. If both words are the same length and `t` never comes up short, the bag ends up empty, so it is an anagram.

Steps:

1. If the lengths differ, return `false`.
2. Count each letter in `s`.

3. For each letter in `t`, if its count is 0 or missing, return `false`. Otherwise subtract 1.
4. Return `true`.

```
var isAnagram = function(s, t) {
    if (s.length !== t.length) return false;

    const counts = {};
    for (const ch of s) {
        counts[ch] = (counts[ch] || 0) + 1;
    }

    for (const ch of t) {
        if (!counts[ch]) return false;
        counts[ch]--;
    }
    return true;
};
```

Complexity: O(n) time. O(1) space for lowercase letters (26 keys at most), O(k) for k distinct characters in general.

Watch out: the length check is what makes the final `return true` safe. Without it, `s = "aab"`, `t = "ab"` wrongly returns `true`, because `t` fits inside `s` and the leftover `a` is never checked. When `t` is the longer one, the code catches it anyway.

3. Two Sum

Pattern: Seen before? (complement Map)

Like you're 10: At a party, dance partners add up to the target. Each person who walks in asks, "Is my partner already here?" If yes, you've found the pair. If not, they write their number and seat in the guest book so the next person can find them.

Steps:

1. Make an empty Map of value → index.
2. For each number, work out `need = target - num`.
3. If the Map has `need`, return `[its index, current index]`.
4. Otherwise store the current number and its index.

```
var twoSum = function(nums, target) {
    const seen = new Map();   // value -> index
    for (let i = 0; i < nums.length; i++) {
        const need = target - nums[i];
        if (seen.has(need)) return [seen.get(need), i];
        seen.set(nums[i], i);
    }
    return [];
};
```

Complexity: O(n) time, O(n) space. Brute force with two nested loops is O(n²).

Watch out: loop with `i < nums.length`, not `<=`. Check before you store, or a number can match itself (for example `[3]` with target 6). Declare `need` with `const`.

Reusable template (fill in the five blanks before coding):

Blank	Two Sum	Contains Duplicate
1. Look for	`target - nums[i]`	`nums[i]`
2. Return when found	`[seen.get(need), i]`	`true`
3. Store	`nums[i]`	`nums[i]`
4. Remember about it	`i`	nothing (Set)
5. Not found	`[]`	`false`

4. Group Anagrams

Pattern: Group by a key (Map of lists)

Like you're 10: Every word lines its letters up alphabetically, and that becomes its locker number. `"eat"`, `"tea"` and `"ate"` all become `"aet"`, so they share a locker. At the end, you hand back what's in every locker.

Steps:

1. Make an empty object (or Map) of key → list of words.
2. For each word, build its key by sorting its letters.
3. Add the word to that key's list, creating the list the first time.
4. Return all the lists with `Object.values`.

```
var groupAnagrams = function(strs) {
    const groups = {};
    for (const word of strs) {
        const key = word.split('').sort().join('');
        if (!groups[key]) groups[key] = [];
        groups[key].push(word);
    }
    return Object.values(groups);
};
```

Trace with `["eat","tea","tan","ate","nat","bat"]`: keys `aet`, `aet`, `ant`, `aet`, `ant`, `abt` → `[[eat,tea,ate],[tan,nat],[bat]]`.

Complexity: O(n·k log k) time because each word is sorted, O(n·k) space.

Watch out: sort the current word (`word.split`), not the whole list (`strs.split` crashes, because arrays have no `.split`). Name the loop variable `word`, not `ch`, so that mix-up stands out. Use `{}` or a Map, not `[]`, for string keys. Remember to return.

Follow-up: to drop the sort, build the key from letter counts instead: 26 zeros, count each letter, join them like `"1#0#0#…"`. That makes it O(n·k).

5. Top K Frequent Elements

Pattern: Count, then rank

Like you're 10: The question is "Who are the top 2 scorers?", not "Who scored exactly 2 goals?" First you tally everyone's goals. Then you line them up from most goals to fewest and take the first k.

Steps:

1. Count each number in a Map.
2. Turn the Map into `[number, count]` pairs.
3. Sort the pairs by count, biggest first.
4. Take the first k pairs and return just the numbers.

```
var topKFrequent = function(nums, k) {
    const counts = new Map();
    for (const num of nums) {
        counts.set(num, (counts.get(num) || 0) + 1);
    }

    const pairs = [...counts.entries()];      // [number, count]
    pairs.sort((a, b) => b[1] - a[1]);         // biggest count first
    return pairs.slice(0, k).map(pair => pair[0]);
};
```

Complexity: O(n log n) time because of the sort, O(n) space.

Faster follow-up (bucket sort, O(n)): a number can appear at most n times, so make n + 1 buckets where bucket `c` holds every number that appears `c` times. Then walk the buckets from the highest count down until you have k numbers.

```
var topKFrequent = function(nums, k) {
    const counts = new Map();
    for (const num of nums) {
        counts.set(num, (counts.get(num) || 0) + 1);
    }

    const buckets = Array.from({ length: nums.length + 1 }, () => []);
    for (const [num, count] of counts) {
        buckets[count].push(num);
    }

    const result = [];
    for (let c = buckets.length - 1; c > 0 && result.length < k; c--) {
        result.push(...buckets[c]);
    }
    return result.slice(0, k);
};
```

Watch out: `k` is how many numbers to return, not a count to match. With a Map, use only `.get` / `.set`; `map[num] = …` tapes a property to the outside of the Map instead of storing an entry. `Object.entries` on a plain object gives string keys (`"1"`), while a Map keeps them as numbers. `b - a` sorts biggest first.

6. Encode and Decode Strings

Pattern: Length-prefix encoding

Like you're 10: Each package gets a label that says "the next 5 items belong to me." The reader reads the label, counts out exactly 5 items without looking at what they are, then reads the next label. That's why a `#` inside a word can't confuse it.

LeetCode 271 is Premium-only. Practice it free on NeetCode.

Steps:

1. Encode: for each string, write its length, then `#`, then the string.
2. Decode: keep a pointer `i`. Walk forward from `i` to the next `#`; everything before it is the length.
3. Take exactly that many characters right after the `#`.
4. Move `i` past them and repeat until the end.

```
class Solution {
    encode(strs) {
        const parts = [];
        for (const s of strs) {
            parts.push(String(s.length), '#', s);
        }
        return parts.join('');
    }

    decode(str) {
        const res = [];
        let i = 0;
        while (i < str.length) {
            let j = i;
            while (j < str.length && str[j] !== '#') j++;
            const length = parseInt(str.substring(i, j));
            i = j + 1;
            res.push(str.substring(i, i + length));
            i += length;
        }
        return res;
    }
}
```

Trace: `["neet", "co#de", ""]` encodes to `"4#neet5#co#de0#"`. Decode reads 4 → `neet`, 5 → `co#de`, 0 → `""`.

Complexity: O(m) time and space, where m is the total number of characters.

Watch out: read every digit up to `#`, not just one, because a 12-character word encodes as `12#…`. A plain separator like a comma breaks as soon as a word contains a comma. Remember to return from encode.

7. Valid Palindrome

Pattern: Two pointers

Like you're 10: Two friends start at opposite ends of a sentence and walk toward each other. Each one skips spaces and punctuation. At every step they compare letters, ignoring capitals. If they ever disagree, it's not a palindrome. If they meet in the middle, it is.

Steps:

1. Put `left` at the start and `right` at the end.
2. Move `left` forward past anything that isn't a letter or digit; do the same for `right`, moving backward.
3. Compare the two characters in lowercase. If they differ, return `false`.
4. Move both pointers inward and repeat until they meet.

```
var isPalindrome = function(s) {
    const isAlnum = ch => /[a-z0-9]/i.test(ch);
    let left = 0;
    let right = s.length - 1;

    while (left < right) {
        while (left < right && !isAlnum(s[left])) left++;
        while (left < right && !isAlnum(s[right])) right--;

        if (s[left].toLowerCase() !== s[right].toLowerCase()) return false;
        left++;
        right--;
    }
    return true;
};
```

Trace: `"A man, a plan, a canal: Panama"` compares `a/a`, `m/m`, `a/a`, `n/n` … and the pointers meet with no mismatch, so it returns `true`.

Complexity: O(n) time, O(1) space.

Watch out: keep `left < right` in the inner skip loops too, or a string of only punctuation runs past the ends. The shortcut of cleaning the string first and comparing it with its reverse also works, but uses O(n) extra space.

8. Product of Array Except Self

Pattern: Prefix / suffix

Like you're 10: Kids stand in a line, each holding a number. One helper walks from the front telling each kid "here's everyone in front of you multiplied." Another walks from the back saying "here's everyone behind you multiplied." Each kid multiplies the two messages, and nobody needed division.

Steps:

1. Build `left[i]`: the product of everything before i (`left[0] = 1`).
2. Build `right[i]`: the product of everything after i (`right[last] = 1`).
3. The answer at i is `left[i] * right[i]`.

```
var productExceptSelf = function(nums) {
    const n = nums.length;
    const left = new Array(n).fill(1);
    const right = new Array(n).fill(1);

    for (let i = 1; i < n; i++) {
        left[i] = left[i - 1] * nums[i - 1];
    }
    for (let i = n - 2; i >= 0; i--) {
        right[i] = right[i + 1] * nums[i + 1];
    }

    const result = [];
    for (let i = 0; i < n; i++) {
        result[i] = left[i] * right[i];
    }
    return result;
};
```

Trace with `[1, 2, 3, 4]`:

index	0	1	2	3
nums	1	2	3	4
left (everything before)	1	1	2	6
right (everything after)	24	12	4	1
answer	24	12	8	6

Complexity: O(n) time, O(n) extra space.

Follow-up (O(1) extra space, output array not counted): write the left products straight into the result, then walk back from the end with one running number for the right side.

```
var productExceptSelf = function(nums) {
    const n = nums.length;
    const result = new Array(n).fill(1);

    for (let i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }

    let rightProduct = 1;
    for (let i = n - 1; i >= 0; i--) {
        result[i] *= rightProduct;
        rightProduct *= nums[i];
    }
    return result;
};
```

Watch out: declare the loop counter with `let` in every loop. Without it, all three loops share one global `i`.

9. Valid Sudoku

Pattern: Seen before? across several groups

Like you're 10: Every cell belongs to three clubs: its row, its column and its 3×3 box. Before a number joins, you check all three member lists. If any club already has that number, the board is invalid.

Steps:

1. Make 9 Sets for rows, 9 for columns and 9 for boxes.
2. Skip `'.'` cells.
3. Work out the box number: `Math.floor(row/3) * 3 + Math.floor(col/3)`.
4. If the row, column or box Set already has the value, return `false`; otherwise add it to all three.

```
var isValidSudoku = function(board) {
    const rows = Array.from({ length: 9 }, () => new Set());
    const cols = Array.from({ length: 9 }, () => new Set());
    const boxes = Array.from({ length: 9 }, () => new Set());

    for (let r = 0; r < 9; r++) {
        for (let c = 0; c < 9; c++) {
            const cell = board[r][c];
            if (cell === '.') continue;

            const b = Math.floor(r / 3) * 3 + Math.floor(c / 3);
            if (rows[r].has(cell) || cols[c].has(cell) || boxes[b].has(cell))
{
                return false;
            }
            rows[r].add(cell);
            cols[c].add(cell);
            boxes[b].add(cell);
        }
    }
    return true;
};
```

Box numbering (boxes are numbered like theater seats, 3 per row):

cell (r, c)	box row	box col	box number
(0, 0)	0	0	0
(1, 5)	0	1	1
(4, 4)	1	1	4
(8, 7)	2	2	8

Complexity: O(1) time and space, because the board is always 81 cells.

Warm-up you did first: LeetCode 2133, Check if Every Row and Column Contains All Numbers, is the same pattern without boxes. **Watch out:** keep the `has` / `add` checks inside the inner loop, where `row` and `col` exist. The `row * width + col` trick for turning a 2D position into one number shows up in images, game boards and matrix storage.

10. Longest Consecutive Sequence

Pattern: Streak start (Set)

Like you're 10: Kids hold numbered cards and stand scattered around a room. Only a kid with nobody holding the number just before theirs starts counting. That kid calls "next number?" and walks forward until the chain breaks. Everyone else waits, so no chain is counted twice.

Steps:

1. Put every number in a Set (instant lookups, duplicates gone).
2. For each number in the Set, check whether `num - 1` is missing. If it is, this number starts a streak.
3. From a start, count forward while `num + length` is in the Set.
4. Keep the longest length seen.

```
var longestConsecutive = function(nums) {
    const set = new Set(nums);
    let best = 0;

    for (const num of set) {
        if (!set.has(num - 1)) {            // num starts a streak
            let length = 1;
            while (set.has(num + length)) { // next number exists
                length++;
            }
            best = Math.max(best, length);
        }
    }
    return best;
};
```

Trace with `[100, 4, 200, 1, 3, 2]`: starts are 100 (length 1), 200 (length 1) and 1 (2, 3, 4 follow, length 4). 4, 3 and 2 are skipped because the number before each is present. Answer: 4.

Complexity: O(n) time, because the inner loop only runs from streak starts, so each number is walked once in total. O(n) space. Sorting would be O(n log n).

Watch out: call `.has` on the Set, never on `num` (a number has no `.has`). Before running, read each `.has(...)` and say what you're searching in. Loop over the Set, not `nums`, so duplicate start numbers don't re-walk the same streak.

Recurring mistakes from this session

Three slips came up across several problems. Scan for them before every Run.

Mistake	Where it showed up	The check
Variable with no `let` / `const`	Two Sum, Valid Anagram, Group Anagrams, Encode, Product Except Self	Every name on the left of `=` the first time gets `let` or `const`.
Method doesn't match the data structure	Top K (`map[num]` on a Map), Group Anagrams (`strs.split`), Longest Consecutive (`num.has`)	Map uses `.get` / `.set`, objects use `[ ]`, only Sets and Maps have `.has`, only strings have `.split`.
Checking the wrong thing	Contains Duplicate (index instead of value), check outside the inner loop (row/column problem)	Say out loud what each `has` searches in and what it searches for.

Naming loop variables for what they hold (`word`, `num`, `cell`) makes the second and third mistakes easier to spot.

Top resources with reasoning

1. NeetCode 150 roadmap: every problem here has a free page and a video that draws the pattern before coding. Watch the video only after your own attempt.
2. NeetCode: String Encode and Decode: the free version of Premium-only LeetCode 271.

3. MDN: Map and MDN: Set: the methods behind seven of the ten problems, and the Map-vs-object differences that caused the Top K bug.
4. MDN: Array.prototype.sort: explains the comparator (`a - b` vs `b - a`) used in Top K and Group Anagrams.
5. LeetCode 2133: Check if Every Row and Column Contains All Numbers: the stepping stone to Valid Sudoku.